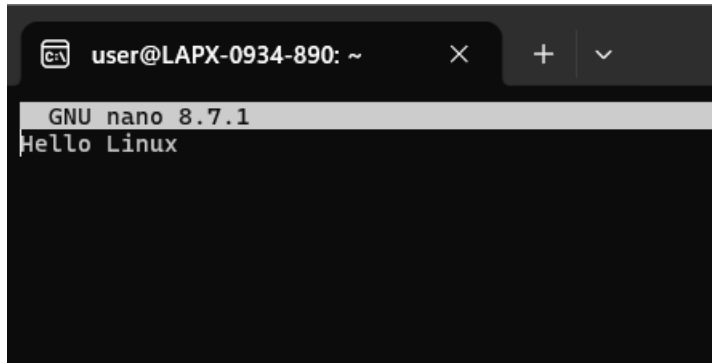


OSSP PRACTICAL WEEK-10

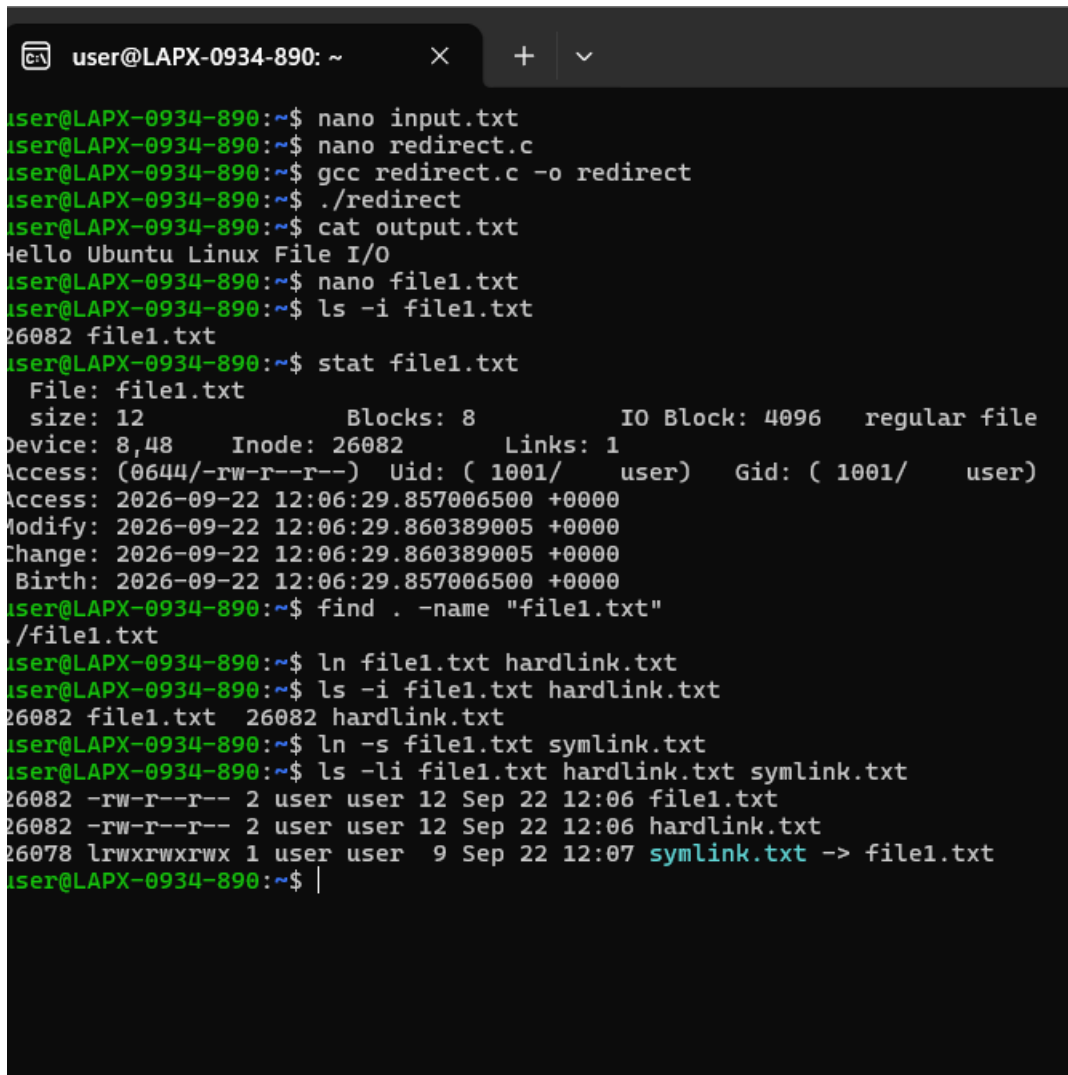
TASK-1: Investigate inode structures using commands such as `ls -li`, `stat`, and `find`. Create hard links and symbolic links and explain their impact on inode allocation.

SOURCE CODE:



```
GNU nano 8.7.1
Hello Linux
```

OUTPUT:



```
user@LAPX-0934-890: ~$ nano input.txt
user@LAPX-0934-890: ~$ nano redirect.c
user@LAPX-0934-890: ~$ gcc redirect.c -o redirect
user@LAPX-0934-890: ~$ ./redirect
user@LAPX-0934-890: ~$ cat output.txt
Hello Ubuntu Linux File I/O
user@LAPX-0934-890: ~$ nano file1.txt
user@LAPX-0934-890: ~$ ls -li file1.txt
26082 file1.txt
user@LAPX-0934-890: ~$ stat file1.txt
  File: file1.txt
  size: 12          Blocks: 8          IO Block: 4096   regular file
Device: 8,48   Inode: 26082    Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1001/   user)   Gid: ( 1001/   user)
Access: 2026-09-22 12:06:29.857006500 +0000
Modify: 2026-09-22 12:06:29.860389005 +0000
Change: 2026-09-22 12:06:29.860389005 +0000
 Birth: 2026-09-22 12:06:29.857006500 +0000
user@LAPX-0934-890: ~$ find . -name "file1.txt"
./file1.txt
user@LAPX-0934-890: ~$ ln file1.txt hardlink.txt
user@LAPX-0934-890: ~$ ls -li file1.txt hardlink.txt
26082 file1.txt 26082 hardlink.txt
user@LAPX-0934-890: ~$ ln -s file1.txt symlink.txt
user@LAPX-0934-890: ~$ ls -li file1.txt hardlink.txt symlink.txt
26082 -rw-r--r-- 2 user user 12 Sep 22 12:06 file1.txt
26082 -rw-r--r-- 2 user user 12 Sep 22 12:06 hardlink.txt
26078 lrwxrwxrwx 1 user user  9 Sep 22 12:07 symlink.txt -> file1.txt
user@LAPX-0934-890: ~$ |
```

OBSERVATION: Hard link shares the same inode as the original file, whereas a symbolic link has a separate inode and stores the path of the original file.

TASK-2: Design a program using `mmap()` to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and implementation complexity.

SOURCE CODE:

```
user@LAPX-0934-890: ~  
GNU nano 8.7.1 mmap.c *  
#include <stdio.h>  
#include <fcntl.h>  
#include <unistd.h>  
#include <sys/mman.h>  
#include <string.h>  
  
int main()  
{  
    int fd;  
    char *data;  
    const char *text = "Hello from mmap";  
  
    fd = open("mmapfile.txt", O_RDWR | O_CREAT, 0644);  
  
    if (fd < 0)  
    {  
        perror("open");  
        return 1;  
    }  
  
    ftruncate(fd, 100);  
  
    data = mmap(NULL, 100, PROT_READ | PROT_WRITE,  
                MAP_SHARED, fd, 0);  
  
    if (data == MAP_FAILED)  
    {  
        perror("mmap");  
        close(fd);  
        return 1;  
    }  
  
    strcpy(data, text);  
  
    printf("Data written using mmap: %s\n", data);  
  
    munmap(data, 100);  
    close(fd);  
  
    return 0;  
}
```

OUTPUT:

```
user@LAPX-0934-890:~$ nano mmap.c  
user@LAPX-0934-890:~$ gcc mmap.c -o mmap  
user@LAPX-0934-890:~$ ./mmap  
Data written using mmap: Hello from mmap  
user@LAPX-0934-890:~$ cat mmapfile.txt  
Hello from mmapuser@LAPX-0934-890:~$ |
```

OBSERVATION: The file was successfully mapped into memory using mmap(). Data was written directly to the mapped memory and stored in the file.